Colegio Tecnológico UTN · Diseño Backend · 5.° año · Comisión Berdichevsky

Práctica — El backend de la tienda

Unidad 7 · Clase 2 · Catálogo, usuarios y compras: los cinco huecos de la plantilla

En la clase pasada dejaste lista la base de tu tienda y la plantilla corriendo. Hoy completás el backend: el catálogo, el registro y el login de usuarios, el control de quién compra y la compra completa. Al terminar, tu API va a responder todo lo que el front necesita, y lo vas a comprobar con siete pruebas en Thunder Client.

Seguimos con la librería como ejemplo. El código es el mismo para cualquier rubro: lo que cambia son los datos de tu base.

Paso 0 — Los cinco huecos

Los controllers y las rutas ya vienen hechos. Hoy trabajás en cinco funciones, y cada una responde **501 Falta implementar** hasta que la completes:

Paso	Función	Archivo	Qué hace
1	obtenerProductos	models/productosModel.js	el catálogo
2	registrar	models/authModel.js	crea usuarios
3	iniciarSesion	models/authModel.js	devuelve el token
4	verificarUsuario	middlewares/verificarUsuario.js	controla quién compra
5	crearPedido	models/pedidosModel.js	registra la compra

Cada hueco viene así. Para completarlo, borrá las dos líneas del TODO y escribí el código del paso:

```
async function obtenerProductos(categoriaId) {
  // TODO (Clase 2 · Paso 1): completar con el código de la práctica.
  throw faltaImplementar('obtenerProductos', 'Clase 2 · Paso 1');
}
```

> ⚠ **Un paso a la vez.**
>
> Después de cada paso, reiniciá el servidor (Ctrl + C y npm start) y hacé la prueba de ese paso antes de pasar al siguiente. Si algo falla, sabés exactamente dónde está el problema.

Paso 1 — El catálogo: obtenerProductos

El catálogo trae los productos activos con el nombre de su categoría. Si llega una categoría, filtra por ella. Completá la función en **models/productosModel.js**:

```
async function obtenerProductos(categoriaId) {
  const columnas = 'id, nombre, descripcion, precio, stock,' +
                   ' imagen_url, categorias(nombre)';
  let consulta = supabase
    .from('productos')
    .select(columnas)
    .eq('activo', true)
    .order('nombre');

  if (categoriaId) {
    consulta = consulta.eq('categoria_id', categoriaId);   // filtro opcional
  }

  const { data, error } = await consulta;
  if (error) throw error;
  return data;
}
```

¿Qué hace cada parte?

Código	Qué hace
`categorias(nombre)`	sigue la clave foránea y trae el nombre de la categoría: es el JOIN de SQL, armado por Supabase
`.eq('activo', true)`	solo muestra los productos que están a la venta
`.order('nombre')`	los ordena alfabéticamente: es el ORDER BY de SQL
`let consulta`	con let, y no const, porque la consulta se arma de a partes: el filtro se suma solo si vino
`req.query.categoria`	en el controller (ya hecho): GET /productos?categoria=2 llega como req.query.categoria

Probalo: en Thunder Client, **GET http://localhost:3000/productos**. Tendrías que recibir tus productos con su categoría adentro:

```
{
  "id": 1,
  "nombre": "El Eternauta",
  "descripcion": "Edición integral",
  "precio": 32000,
  "stock": 6,
  "imagen_url": "https://xxxx.supabase.co/storage/v1/object/public/productos/1.jpg",
  "categorias": { "nombre": "Novelas" }
}
```

Probá también **GET /productos?categoria=1**: tienen que llegar solo los de la primera categoría.

Paso 2 — El registro: registrar

Nunca guardamos contraseñas en nuestras tablas. **Supabase Auth** crea el usuario en auth.users y guarda la contraseña cifrada. Nosotros, con el mismo id que nos da Auth, guardamos el nombre en perfiles. Completá la función en **models/authModel.js**:

```
async function registrar(email, password, nombre) {
  const { data, error } = await supabase.auth.admin.createUser({
    email,
    password,
    email_confirm: true          // no pedimos confirmar por mail
  });
  if (error) throw error;

  const { error: e2 } = await supabase.from('perfiles')
    .insert({ id: data.user.id, nombre });
  if (e2) throw e2;
  return data.user;
}
```

Código	Qué hace
`auth.admin.createUser`	crea el usuario en auth.users; solo funciona con la clave secreta
`email_confirm: true`	lo da por confirmado, así no hace falta configurar el envío de mails
`data.user.id`	el id que le dio Auth: lo reusamos en perfiles para que las dos tablas queden unidas
`{ error: e2 }`	renombra el error de la segunda consulta para no pisar el de la primera

Probalo: POST /registro con este body. Tiene que responder 201:

```
{ "email": "ana@mail.com", "password": "123456", "nombre": "Ana" }
```

✓ **Comprobalo en el panel.**

En Supabase, **Authentication → Users** tiene que aparecer ana@mail.com. Y en la tabla **perfiles**, una fila con el nombre Ana y el rol cliente.

Paso 3 — El login: iniciarSesion

Cuando el usuario inicia sesión, Auth controla la contraseña y nos devuelve un **token**. El token funciona como la pulsera de un recital: te la dan al entrar y la mostrás en cada puerta, sin volver a dar la contraseña. Completá la segunda función de **models/authModel.js**:

```
async function iniciarSesion(email, password) {
  // un cliente descartable: la sesión queda acá y no en el cliente principal
  const temporal = createClient(
    process.env.SUPABASE_URL,
    process.env.SUPABASE_KEY,
    { auth: { persistSession: false } }
  );
  const { data, error } =
    await temporal.auth.signInWithPassword({ email, password });
  if (error) return null;
  return data.session.access_token;
}
```

> ⚠ **¿Por qué un cliente nuevo y no el de siempre?**
> El cliente con el que se hace el login queda "logueado" como ese usuario. Si fuera el cliente principal, todo el servidor pasaría a trabajar con los permisos de un cliente y dejaría de usar la clave secreta. El temporal se usa una sola vez y se descarta.

Código	Qué hace
`signInWithPassword`	Auth compara el email y la contraseña con los que tiene guardados
`return null`	los datos no coinciden: el controller (ya hecho) responde 401 "Email o contraseña incorrectos"
`access_token`	el token: el front lo guarda y lo manda en cada compra

Probalo: POST /login con el email y la contraseña de Ana. Tiene que responder 200 con un token largo. **Copialo**: lo vas a usar en los pasos 4 y 6.

```
{ "token": "eyJhbGciOiJIUzI1NiIsImtpZCI6..." }
```

Paso 4 — El control de la puerta: verificarUsuario

La ruta de compra está protegida por un **middleware**, igual que el logger de la Unidad 5: va entre la ruta y el controller. Lee el token del header, le pregunta a Auth de quién es y, si todo está bien, deja el usuario en req.usuario. Completá **middlewares/verificarUsuario.js**:

```
async function verificarUsuario(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.replace('Bearer ', '');
  if (!token) return res.status(401).send('Tenes que iniciar sesion');

  const { data, error } = await supabase.auth.getUser(token);
  if (error || !data.user) return res.status(401).send('Sesion invalida');

  req.usuario = data.user;     // desde acá, el controller sabe quién compra
  next();
}
```

Código	Qué hace
`req.headers.authorization`	el header donde viaja el token: "Bearer eyJhbGci..."
`.replace('Bearer ', '')`	le saca la palabra Bearer y deja solo el token
`auth.getUser(token)`	le pregunta a Auth de quién es ese token y si sigue vigente
`req.usuario`	el usuario queda guardado en la petición para el controller
`next()`	deja pasar al controller; sin next(), la petición se queda colgada

> ⚠ **401 y 403 no son lo mismo.**
> **401**: no sabemos quién sos (no hay token, o no sirve). **403**: sabemos quién sos, pero no tenés permiso. El 403 lo vas a usar si hacés el extra del panel admin.

Probalo: POST /pedidos sin token tiene que dar 401. Ahora pegá el token en la pestaña **Auth** de Thunder Client, eligiendo **Bearer**, y mandalo de nuevo: tiene que dar **501 Falta implementar: crearPedido**. Eso significa que el middleware te dejó pasar.

Paso 5 — La compra: crearPedido

Es la función más importante de la tienda. El backend hace cinco cosas en orden: busca los productos, valida, calcula el total con los precios de la base, guarda el pedido con sus items y descuenta el stock. Si algo no sirve, corta **antes** de escribir en la base.

> ⚠ **errorCliente ya está en la plantilla.**
> Está arriba de crearPedido, en el mismo archivo. Pegá solo el **cuerpo** de crearPedido: si volvés a escribir la función errorCliente, Node avisa "Identifier 'errorCliente' has already been declared".

```
async function crearPedido(usuarioId, items) {
  // 1. Buscamos todos los productos del carrito en una sola consulta
  const ids = items.map(i => i.producto_id);
  const { data: productos, error: e1 } = await supabase
    .from('productos')
    .select('id, nombre, precio, stock')
    .in('id', ids)
    .eq('activo', true);
  if (e1) throw e1;

  // 2 y 3. Validamos y calculamos el total con precios de la base
  let total = 0;
  const filas = [];
  for (const item of items) {
    const p = productos.find(x => x.id === item.producto_id);
    if (!p) throw errorCliente('No existe el producto ' + item.producto_id);
    if (!Number.isInteger(item.cantidad) || item.cantidad < 1) {
      throw errorCliente('Cantidad invalida para ' + p.nombre);
    }
    if (p.stock < item.cantidad) throw errorCliente('Sin stock de ' + p.nombre);
    total += p.precio * item.cantidad;
    filas.push({
      producto_id: p.id,
      cantidad: item.cantidad,
      precio_unitario: p.precio      // la foto del precio
    });
  }

  // 4. Guardamos el pedido y después sus items
  const { data: pedido, error: e2 } = await supabase
    .from('pedidos')
    .insert({ usuario_id: usuarioId, total })
    .select()
    .single();
  if (e2) throw e2;

  const conPedido = filas.map(f => ({ ...f, pedido_id: pedido.id }));
  const { error: e3 } = await supabase.from('pedido_items').insert(conPedido);
  if (e3) throw e3;
```

```
  // 5. Descontamos el stock de cada producto
  for (const f of filas) {
    const p = productos.find(x => x.id === f.producto_id);
    await supabase
      .from('productos')
      .update({ stock: p.stock - f.cantidad })
      .eq('id', p.id);
  }

  return pedido;
}
```

¿Qué hace cada parte?

Código	Qué hace
`.in('id', ids)`	trae varios productos de una vez: es el WHERE id IN (1, 3) de SQL
`errorCliente(...)`	un error con status 400: el controller (ya hecho) responde 400 con el mensaje
`p.precio * item.cantidad`	el total sale del precio de la base, nunca de uno que mande el front
`precio_unitario: p.precio`	la foto del precio en el momento de la compra
`pedido.id`	los items lo necesitan: por eso el pedido se guarda primero
`...f`	copia las propiedades del item y le suma pedido_id
`usuarioId`	llega del controller, que lo saca de req.usuario.id: nunca del body

Paso 6 — Las siete pruebas

Con los cinco huecos completos, reinicié el servidor y corré todas las pruebas en este orden. Para las que dicen "con token", usá la pestaña **Auth → Bearer** de Thunder Client.

#	Request	Body (JSON)	Esperado
1	GET /productos	sin body	200 con categorias
2	POST /registro	{ "email": "beto@mail.com", "password": "123456", "nombre": "Beto" }	201
3	POST /login	{ "email": "beto@mail.com", "password": "123456" }	200 y el token
4	POST /pedidos sin token	{ "items": [{ "producto_id": 1, "cantidad": 2 }] }	401
5	POST /pedidos con token	{ "items": [{ "producto_id": 1, "cantidad": 2 }] }	201 y el pedido
6	POST /pedidos con token	{ "items": [{ "producto_id": 6, "cantidad": 99 }] }	400: sin stock
7	POST /login	{ "email": "beto@mail.com", "password": "otra" }	401

✓ Si las siete pruebas dan lo esperado, tu backend está completo.

Después de la prueba 5, mirá el Table Editor: un pedido nuevo con su total, sus items con precio_unitario, y el stock del producto 1 descontado en 2.

Cómo queda todo conectado

```
POST /pedidos
  → verificarUsuario     ¿quién compra? deja el usuario en req.usuario
  → pedidosController    ¿hay carrito?
  → pedidosModel         valida, calcula el total y guarda
  → Supabase             pedidos, pedido_items y el stock de productos
```

Ruta	Pasa por	Termina en
`GET /productos`	productosController → obtenerProductos	productos + categorias
`POST /registro`	authController → registrar	auth.users + perfiles
`POST /login`	authController → iniciarSesion	auth.users (devuelve el token)
`POST /pedidos`	verificarUsuario → pedidosController → crearPedido	pedidos + pedido_items + productos

Errores frecuentes y soluciones

POST /pedidos responde 401 aunque hiciste login

✕ Causa: El token no llega bien: falta "Bearer " adelante, o se venció (dura una hora).

✓ Solución: Usá la pestaña Auth con Bearer. Si pasó mucho tiempo, hacé login de nuevo y copiá el token nuevo.

El registro falla con "User not allowed"

✕ Causa: Estás usando la clave pública: auth.admin solo funciona con la secreta.

✓ Solución: En el .env, SUPABASE_KEY tiene que ser la secret key (o service_role). Reiniciá el servidor después de cambiarlo.

El registro responde "already been registered"

✕ Causa: Ese email ya existe en Authentication.

✓ Solución: Usá otro email para probar, o borrá el usuario desde Authentication → Users.

Identifier 'errorCliente' has already been declared

✕ Causa: Pegaste la función errorCliente otra vez: ya está en la plantilla.

✓ Solución: Borrá la copia que pegaste. Solo va el cuerpo de crearPedido.

El pedido sale con total 0 o NaN

✕ **Causa: La cantidad llega como texto ("2") o con otro nombre de propiedad.**

✓ **Solución: El body tiene que ser { "items": [{ "producto_id": 1, "cantidad": 2 }] }, con números sin comillas.**

Sigue respondiendo 501 aunque completé la función

✕ **Causa: No reiniciaste el servidor, o quedó la línea throw faltaImplementar al principio de la función.**

✓ **Solución: Borrá las dos líneas del TODO y reiniciá con Ctrl + C y npm start.**

Extensión opcional — para los que terminan antes

- Sumá GET /productos/:id para la página de detalle: un model con .eq() y .single(), y respondé 404 si no existe.
- Agregá un buscador: GET /productos?buscar=eter, con .ilike('nombre', '%' + texto + '%').
- Creá GET /mis-pedidos, protegida con verificarUsuario, que devuelva los pedidos de req.usuario.id con sus items: .select('id, total, estado, creado, pedido_items(cantidad, precio_unitario, productos(nombre))').
- **Para pensar:** si dos clientes compran el último producto al mismo tiempo, ¿qué puede pasar con el stock? ¿Entre qué pasos de crearPedido está el problema?

⚠ **Recordá para el proyecto final**

Este es el backend completo de tu tienda: es la mitad del Hito 1. En la Clase 3 conectás el front y los productos aparecen en la página; en la Clase 4 la publicás en Vercel. Si hoy no llegaste a las siete pruebas, terminalas antes de la próxima clase.